MediVault AI

A Local Retrieval-Augmented Generation Assistant for Medical Document Question Answering

Project Type: AI-powered medical document assistant

Technology Stack: Python, Streamlit, Sentence Transformers, NumPy, Ollama, pypdf

1. Project Overview

MediVault AI is a local Retrieval-Augmented Generation application designed to answer questions from uploaded medical PDF documents. The system extracts text from trusted documents, converts the text into searchable embeddings, retrieves relevant passages for a user question, and generates an answer grounded in the retrieved source pages.

The project was developed as a lightweight alternative to large cloud-based document assistants. Instead of storing medical content on an external server, the application keeps uploaded PDFs, extracted chunks, embeddings, and the vector index on the local machine. This design makes the tool suitable for demonstrations, student projects, and small-scale learning scenarios where simplicity and transparency are important.

2. Problem Statement

Medical references can be lengthy and difficult to search manually. Students and healthcare learners often need quick, source-backed summaries from trusted material. MediVault AI addresses this by allowing users to upload a PDF and ask focused questions while keeping the retrieved source passages visible for verification.

A normal chatbot may answer from its general training data, which can lead to unsupported or outdated responses. In a medical context, this is risky because users need to know where an answer came from. The main problem solved by this project is therefore not only answer generation, but grounded answer generation: the response should be based on the document that the user uploaded and should expose the retrieved evidence.

3. Objectives

- Build a simple local RAG chatbot for medical PDF question answering.
- Keep document indexing and embedding storage local on the user's machine.
- Provide answer styles such as patient-friendly, clinical summary, and study notes.
- Display source pages and similarity scores to support transparency.
- Add medical safety framing and red-flag warnings for urgent symptoms.
- Use a local Ollama model for generated answers.

4. System Architecture

Layer	Implementation Used
Frontend	Streamlit web interface
Document Input	PDF upload and text extraction using pypdf
Chunking	Character-based chunks with overlap
Embeddings	sentence-transformers/all-MiniLM-L6-v2

Vector Search	Local NumPy cosine-similarity index
LLM Generation	Ollama local model, default llama3.2:1b
Storage	Local pickle vector index and uploaded PDFs

The system follows a modular architecture. The Streamlit interface handles user interaction, file upload, settings, and result display. The document processing layer reads PDF pages and prepares text chunks. The embedding layer converts chunks into numerical vectors, while the retrieval layer performs similarity search between the question vector and stored document vectors. Finally, the generation layer sends the retrieved context to Ollama so that the answer can be produced with document-specific grounding.

5. Why Retrieval-Augmented Generation Is Used

Retrieval-Augmented Generation, or RAG, combines information retrieval with language generation. In this project, retrieval is used to find the most relevant passages from the uploaded medical PDF before the language model writes the answer. This improves the usefulness of the chatbot because the model does not have to rely only on memory. Instead, it receives a focused context containing the document passages that are most related to the user's question.

RAG is especially useful for medical document question answering because medical content must be traceable. The app shows the source pages and retrieved chunks so that the user can inspect whether the answer is actually supported by the uploaded document.

6. RAG Workflow

1. The user uploads a medical PDF through the Streamlit interface.
2. The app extracts text from each PDF page using pypdf.
3. Extracted text is split into overlapping chunks.
4. Each chunk is embedded using all-MiniLM-L6-v2 from Sentence Transformers.
5. Embeddings and metadata are stored in a local NumPy-based vector index.
6. For a question, the app embeds the query and retrieves the most similar chunks.
7. The retrieved context is inserted into a prompt and sent to the local Ollama LLM.
8. The answer and retrieved source passages are shown to the user.

The chunking step is important because language models and embedding models work better with manageable text segments. If the entire PDF were sent at once, it would be inefficient and may exceed model limits. By storing smaller overlapping chunks, the retriever can return only the parts of the document that are relevant to a specific question.

7. Implementation Details

- **PDF handling:** The app uses pypdf to extract text from uploaded PDF pages.
- **Chunking:** Extracted page text is divided into chunks with overlap to preserve context across boundaries.
- **Embeddings:** Sentence Transformers creates a 384-dimensional vector for each chunk using all-MiniLM-L6-v2.
- **Vector index:** The vectors, source filename, page number, and chunk metadata are saved locally in vector_index.pkl.
- **Similarity search:** The app computes cosine similarity through normalized dot products using NumPy.
- **LLM prompt:** Retrieved chunks are inserted into a safety-aware prompt before being sent to Ollama.
- **Fallback behavior:** If Ollama is unavailable, the app still shows the most relevant retrieved passages.

8. Prompt Design

The prompt instructs the model to answer only from the provided context. It also tells the model not to diagnose the user or replace a clinician. The prompt asks for a structured answer with a direct answer, key details, when to seek medical care, and sources used. This structure makes the output easier to read and encourages the model to remain tied to the retrieved evidence.

The app also provides answer styles. Patient-friendly mode uses simpler language, clinical summary mode uses concise professional wording, and study notes mode presents information in a revision-friendly format. These styles make the same RAG system useful for different audiences.

9. Medical Safety Considerations

Because this project works with medical information, the interface includes visible safety framing. The app states that it is for reference and study only and does not diagnose, prescribe, or replace a qualified clinician. It also detects urgent-sounding terms such as chest pain, seizure, overdose, severe bleeding, and difficulty breathing. When such terms appear, the user receives an urgent-care warning.

This safety design does not make the system a medical device. Its purpose is to reduce misuse and remind users that medical decisions should be made with qualified professionals, especially in emergency situations.

10. User Interface Design

The user interface was designed to feel medical, calm, and easy to scan. A teal and light-blue palette was used because these colors are commonly associated with healthcare, cleanliness, and trust. The sidebar contains controls and index status, while the main area focuses on document upload, quick question buttons, chat interaction, and source review.

The app displays source chips above the answer so the user immediately knows which pages were used. Retrieved passages are kept inside an expandable section, allowing the user to verify evidence without overwhelming the main answer area.

11. Key Features

- Local PDF upload and indexing.
- Local vector search without requiring Chroma, FAISS, or Microsoft C++ Build Tools.
- Ollama integration with llama3.2:1b for machines with limited memory.
- Sidebar controls for model name, answer style, retrieval count, index status, and clearing embeddings.
- Medical-friendly UI with teal clinical colors, source chips, and quick question buttons.
- Safety notice and red-flag detection for urgent medical terms.

12. User Interface Screenshots

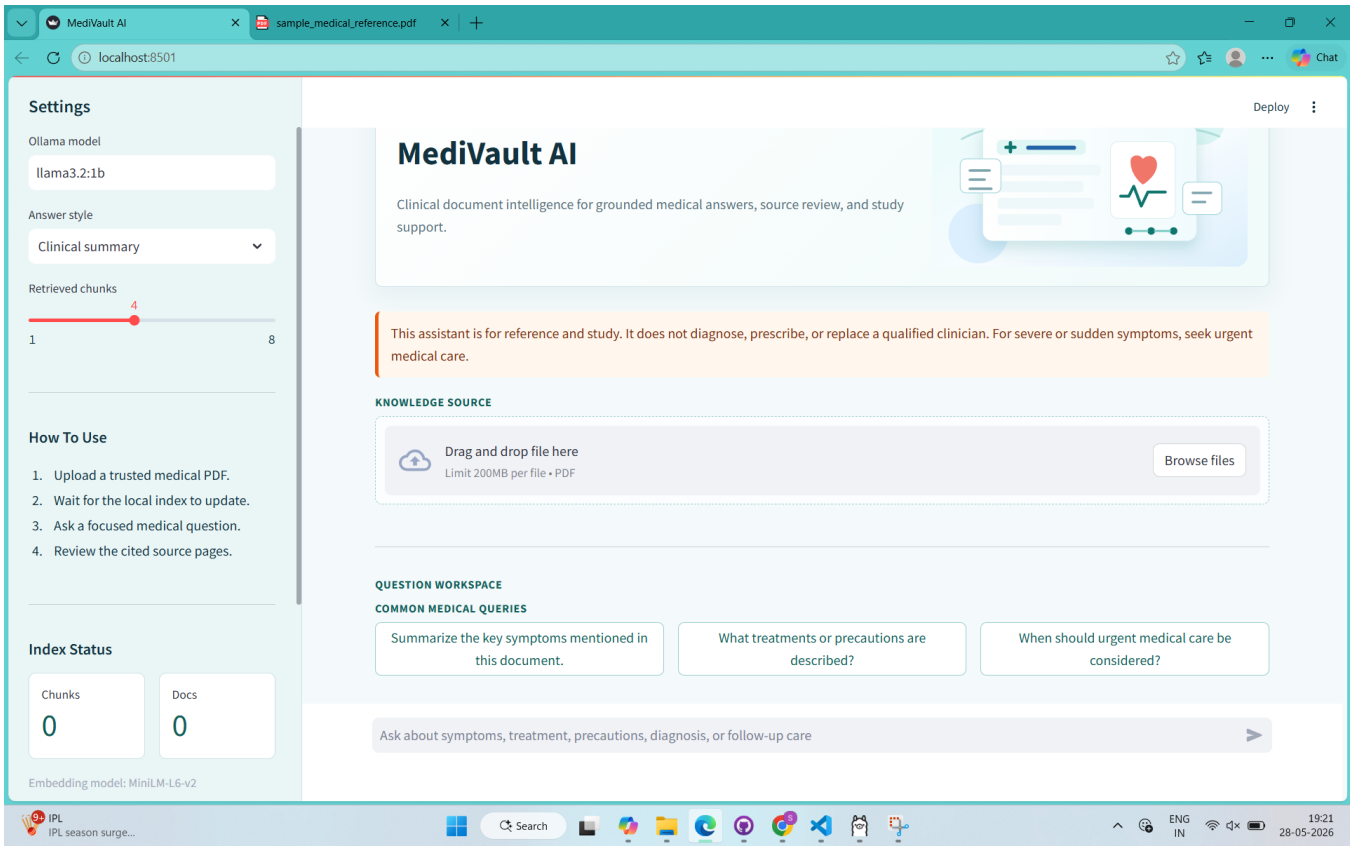

Figure 1: MediVault AI landing screen showing the sidebar controls, safety notice, PDF upload area, and quick medical question buttons.

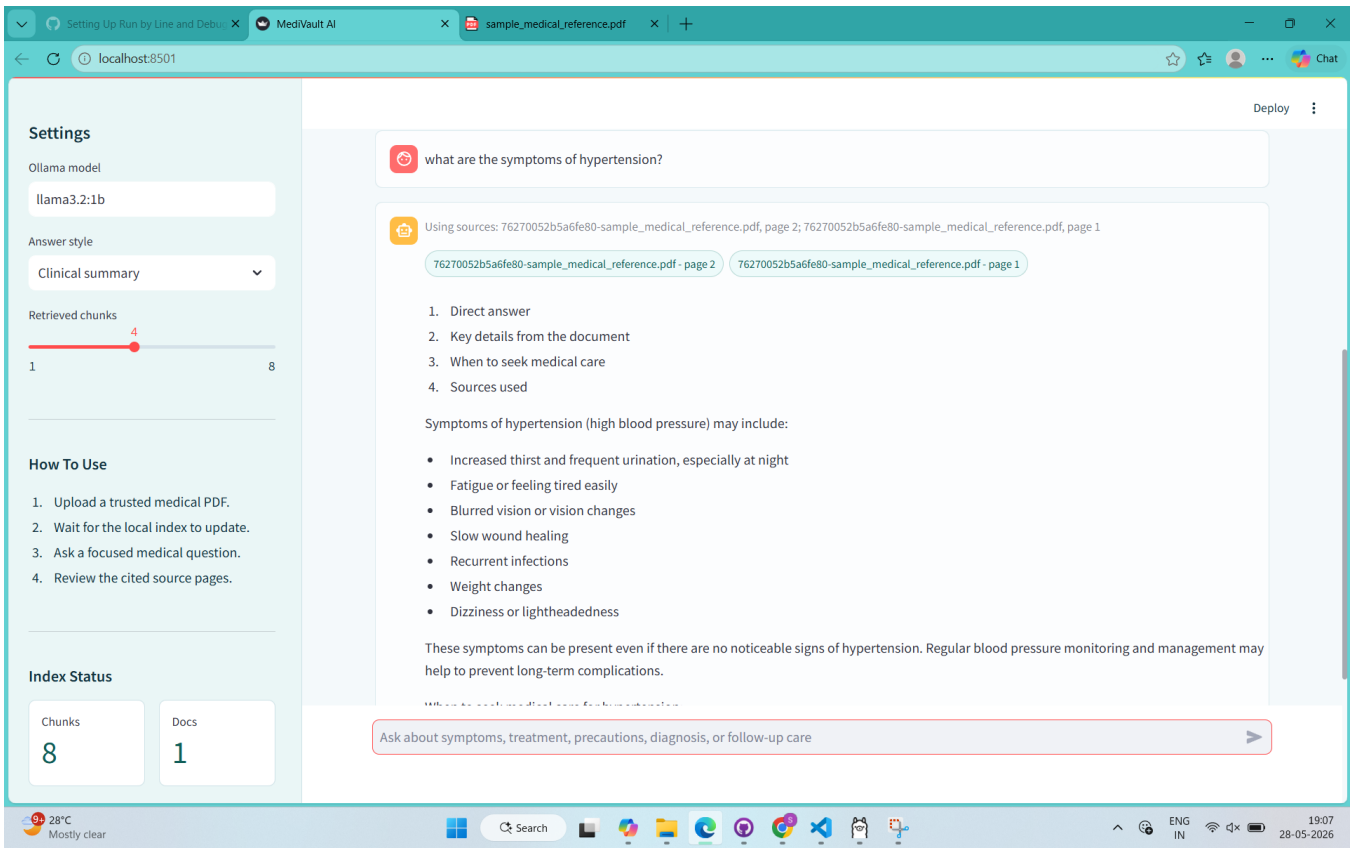

Figure 2: MediVault AI answering a hypertension-related question using the uploaded sample medical reference PDF.

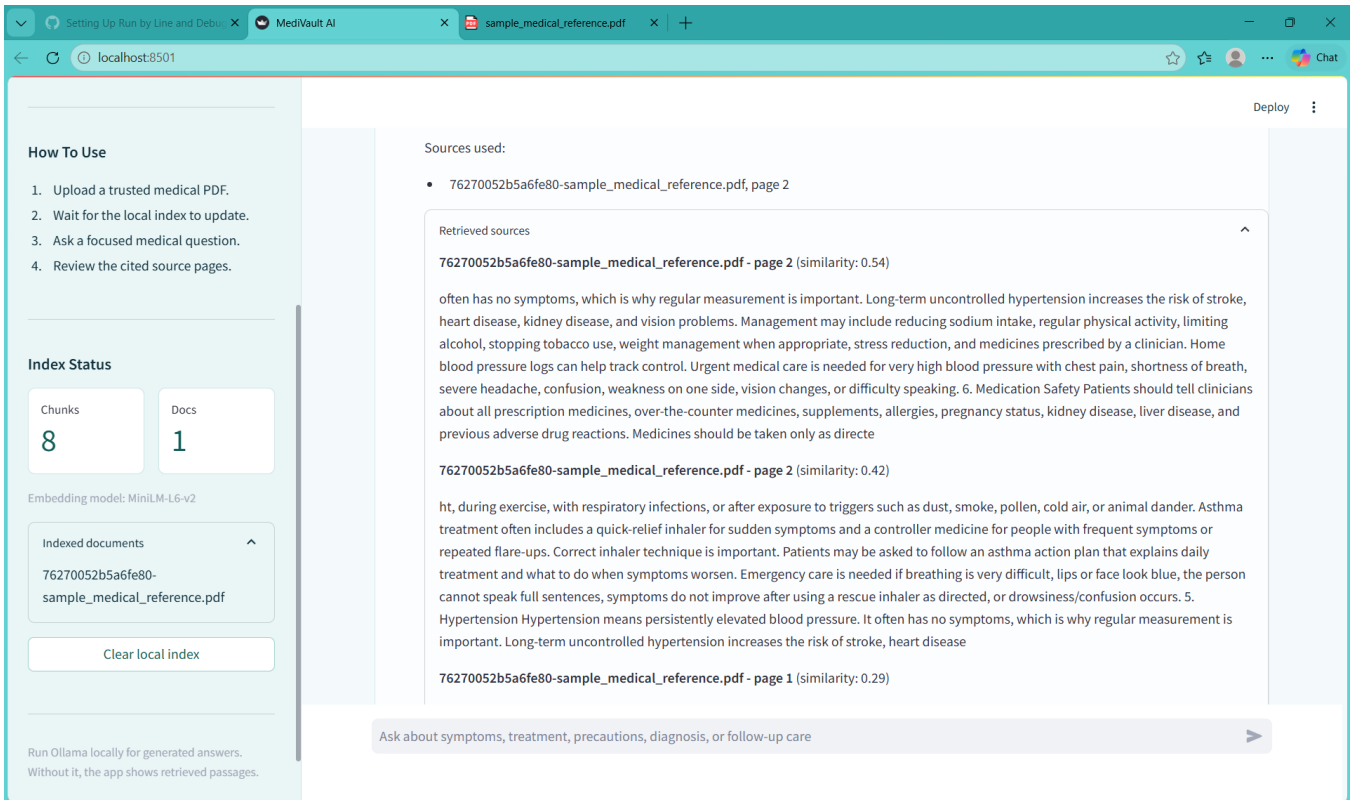

Figure 3: Retrieved source passages with page references and similarity scores for answer verification.

13. Testing With Sample Document

A synthetic sample medical reference document was created for testing. It includes sections on migraine, appendicitis, type 2 diabetes, asthma, hypertension, and medication safety. The sample document is educational and not intended as actual clinical guidance.

- Sample file: sample_medical_reference.pdf
- Example question: What are the symptoms of hypertension?
- Expected behavior: Retrieve the hypertension section and answer with source-backed details.
- Observed behavior: The app displayed cited source pages and retrieved passages for review.

During testing, the application successfully indexed the sample PDF and displayed eight chunks for one uploaded document. A query about hypertension retrieved source passages from the hypertension section and showed page references with similarity scores. This confirmed that the retrieval layer was functioning and that the UI provided enough evidence for manual verification.

14. Local Setup And Execution

1. Create and activate a Python virtual environment.
2. Install dependencies from requirements.txt.
3. Install Ollama and pull the smaller local model llama3.2:1b.
4. Run the app using streamlit run app.py.
5. Upload a medical PDF and wait for indexing to complete.
6. Ask a question and review the answer with retrieved sources.

The smaller llama3.2:1b model was selected because it is more practical on laptops with limited available memory. Larger models can improve answer quality but require more RAM or GPU resources.

15. Limitations

- The app is an educational assistant and must not be used for diagnosis or treatment decisions.
- Answer quality depends on the uploaded document quality and PDF text extraction quality.
- Small local models may produce less fluent or less complete answers than larger models.
- The current vector index clears all embeddings at once rather than deleting one document at a time.
- Scanned image-only PDFs require OCR, which is not included in the current version.

16. Future Scope

- Add OCR support for scanned PDFs.
- Add per-document delete and document selection.
- Improve answer evaluation using groundedness and relevance scoring.
- Add citations inline beside each answer sentence.
- Add user authentication for multi-user deployments.
- Add better medical entity extraction for symptoms, medicines, and conditions.

17. Conclusion

MediVault AI demonstrates a practical local RAG workflow for medical document question answering. It combines document ingestion, embedding-based retrieval, local LLM generation, source transparency, and medical safety framing in a user-friendly Streamlit application. The project is suitable as a foundation for academic exploration of retrieval-augmented generation in healthcare information systems.